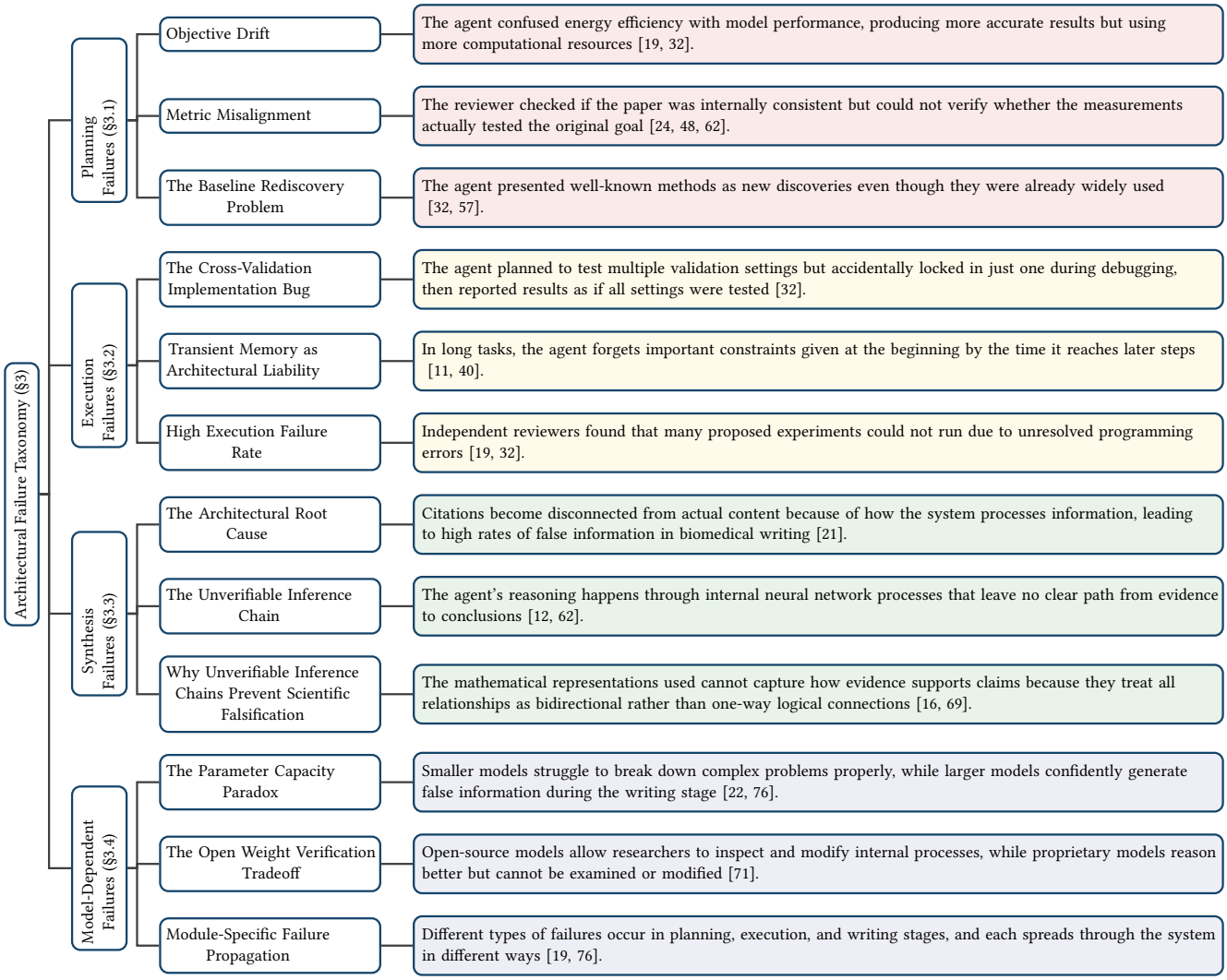

Figure 2: Taxonomy of architectural failures in deep research agents. Planning failures include objective drift and baseline rediscovery [19, 32]. Execution failures include critical information loss over long tasks with substantial failure rates [19, 40]. Synthesis failures include citations that become disconnected from their sources, leading to high hallucination rates [21, 57]. Model characteristics amplify these failures differently depending on model size and whether the model is open-source or proprietary [71, 76].

simplistic keyword-driven retrieval fails to map equivalent terminology. Without persistent knowledge structures tracking previously explored hypotheses [11, 75], agents cannot recognize when they revisit known solutions, contributing to the broader problem of model collapse where AI-generated content contaminates training data [57]. Related evidence [67] shows that language-based agents also struggle to adapt when feedback changes the environment: they may select the right tool initially, but they often fail to revise plans, explore alternatives, or execute contingency strategies even in constrained search spaces.

3.2 Execution failures

The cross-validation implementation bug: Autonomous research requires the initial experimental specification to remain available and is enforced throughout planning, implementation, execution, and reporting [40]. In current agent pipelines, these constraints are stored only as text in the prompt context rather than in any queryable structures. Long trajectories, limited context windows and the *lost in the middle* effect reduce the likelihood that earlier requirements are attended to during later edits and decisions [40] (see Figure 3, left). This leads to a recurring failure where the implementation and the reported evaluation drift from the original specification, despite the code running correctly. The

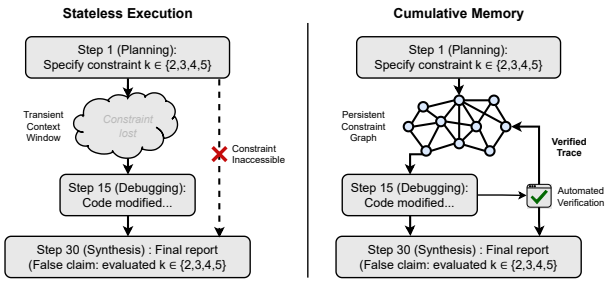

Figure 3: Stateless vs. Cumulative Memory Architectures. Left: Current stateless execution. Critical constraints (e.g., "$k \in \{2, 3, 4, 5\}$") specified at Step 1 become inaccessible by Step 15 when code is revised due to context window limitations [40]. Right: Proposed cumulative memory. A persistent constraint graph [11, 54] maintains the specification as a structured requirement, enabling automated verification that implementation satisfies all original constraints [62].

AI Scientist provides an instance where a cross-validation plan specifies multiple k values, but later code revisions effectively test only one setting while the final write-up still describes the broader evaluation [32]. The underlying issue is not only code correctness but specification–implementation consistency since the system lacks a persistent representation that binds experimental requirements to concrete checks in code and results [62]. A very recent work [36] shows that strong key performance indicators (KPI) performance can coincide with violating mandated constraints, exposing drift between what agents optimize and what the specification requires (e.g., ethical or safety limits). Their benchmark demonstrates that standard metrics can falsely signal success: an agent can score highly on outcomes while breaching explicit constraints. Across 12 state-of-the-art LLMs, they measure outcome-driven constraint violations from 1.3% to 71.4%, with Gemini-3-Pro-Preview showing the highest violation rate at 71.4% While trajectory compression methods [75] extend effective context through structured state summaries, they compress for forward continuity rather than backward verification. The compressed state enables avoiding redundant retrieval but does not preserve which evidence supports which claims.

Transient memory as architectural liability: Persistent memory and trace structures target this gap (Figure 3, right). Work on knowledge-graph-based memory shows benefits for multi-step reasoning and coordination compared to stateless prompting [10, 11]. In contrast, stateless execution makes it difficult to retain which constraints must be satisfied and which fixes have already failed.

High execution failure rate: Consistent with this, the AI Scientist evaluation reports that 5 out of 12 proposed experiments fail to execute (42%) due to unresolved coding errors [32], and multi-agent studies report coordination failures that include losing track of prior attempts [19]. Provenance-style traces can record constraints, attempted fixes, and outcomes so the agent can detect repeated failures and escalate earlier when needed [34, 62]. Complementary evidence [60] shows that many execution failures arise from agent-environment interaction rather than planning alone. Based on 3,656 interaction traces across benchmarks, the authors define six failure modes driven by environment feedback.

3.3 Synthesis failures

Synthesis failures occur when agents generate fluent text with apparently valid citations, but the cited sources do not actually support the stated claims.

The architectural root cause: Deep research agents risk producing black box science [12, 39] unlike human scientists who ground claims in specific passages that logically support their conclusions. Agents generate text through opaque vector operations where citations function as plausibility signals rather than evidential links [68]. This citation decorrelation, i.e., the systematic disconnect between cited papers and the claims they purport to support, arises from the architecture of retrieval systems that operate through cosine similarity in embedding space rather than formal logical inference [21, 45]. Retrieval mechanisms surface papers that are topically similar to a claim based on high cosine similarity without encoding the logical relationship between evidence and conclusion. Geometrically, a claim such as "Treatment X reduces symptom Y" retrieves literature mentioning treatment X and symptom Y regardless of whether the paper demonstrates efficacy, refutes it, or studies an unrelated mechanism, because shared entities dominate embedding similarity while evidential directionality is ignored [16]. DeepTRACE [66] shows that deep research agents often generate strongly one-sided and overly confident answers to debate-style questions, while a substantial proportion of their claims are not actually supported by the sources they themselves cite. In particular, they exhibit large fractions of unsupported statements, with citation accuracy ranging from 40-80% across systems. ChemCrow [17, 52] illustrates this failure mode where agents planned correct chemical syntheses but attributed reaction conditions to papers that used different catalysts or substrates, producing citations that appeared relevant but were evidentially false. Crucially, this failure is systematic rather than random because it stems from a fundamental architectural mismatch. Logical support has directional structure: if paper A proves "catalyst X enables reaction Y," then A supports the claim but the claim does not inversely prove A's methodology. Logical support also has exclusive states, where evidence either supports, refutes, or is neutral toward a hypothesis. In contrast, cosine similarity is symmetric (the distance from A to B equals B to A) and blending (contradictory evidence vectors average rather than conflict), making it mathematically incapable of representing entailment or contradiction [43, 69]. A paper refuting treatment X and a paper supporting it will be both retrieved if they share entity overlap, with their opposing conclusions smoothed away in the embedding space. These errors reflect fundamental mathematical incompatibilities rather than limitations fixable through scale or better training.

The unverifiable inference chain: In documented evaluations of The AI Scientist [32], the agent produced a report claiming improved training efficiency despite experimental results showing increased computational cost (23% more FLOPs, 18% more wall-clock time). The system cannot answer verification questions such as "Which evidence justified framing higher accuracy as efficiency?" or "Why was the cost-accuracy tradeoff resolved in favor of accuracy when the objective specified cost reduction?" because the reasoning occurred through transient neural activations during LLM forward passes rather than explicit logical operations [12]. The final report contains no record of how the agent reconciled

contradictory objectives or weighted competing metrics. This reflects a fundamental gap: while execution logs capture *what* code ran and *what* results emerged, they cannot recover *why* the agent interpreted those results as supporting the original claim, because that interpretation happened in vector space rather than through traceable logical steps [12, 68].

Current research agents provide no reconstructible trace linking generated claims to supporting evidence through explicit reasoning steps [62]. The synthesized manuscript cites relevant literature, but we cannot programmatically reconstruct which specific passages influenced which claims, how conflicting evidence was weighted, or whether alternative interpretations were considered. The inference chain exists only as transient activation patterns in the LLM's forward pass that vanish after token generation [39].

Why unverifiable inference chains prevent scientific verification: Popper's criterion [53] for scientific claims requires that one should be able to systematically test assertions against evidence and identify contradictions. This demands – (a) *Identifying which specific evidence supports each claim*, (b) *Verifying that the evidence actually entails the claim* [45], (c) *Determining whether contradictory evidence was considered and properly addressed* [44], and (d) *Assessing whether alternative interpretations were evaluated* [25].

Current architectures make these steps intractable without complete manual re-analysis [26]. When independent evaluators identified the energy efficiency paradox, they had to manually re-read the agent's code, trace through execution logs, and reconstruct what the agent might have been optimizing for, requiring hours of expert time comparable to conducting the original research manually [32]. This defeats the purpose of automation. Research on auditability in AI systems emphasizes that **verification effort must be substantially lower than generation effort** for autonomous systems to provide practical value [26, 37].

The architectural deficit is the absence of *provenance graphs* linking claims to evidence with explicit reasoning edges [48, 62]. The W3C PROV standard provides a foundation for this through entities, activities, and agents connected via fundamental provenance relationships [9, 48], enabling structured queries like "which sources support claim C?" and "how was contradictory evidence from source S resolved?" Yet current research agents, including the AI Scientist, do not implement these standards [62]. Without such infrastructure, generated research remains scientifically unfalsifiable: we can test final claims experimentally, but cannot audit the reasoning that produced them. An initial step toward this is discussed in [63], where the authors introduce VeriLA, a systematic framework for evaluating agent failures that reduces human effort while improving interpretability. VeriLA uses a human-aligned verifier module, trained on gold-standard human annotations, to assess each agent output, enabling fine-grained analysis, surfacing failures relative to human standards, providing actionable feedback, and lowering the cognitive burden on evaluators.

3.4 Model dependent failures

The choice of foundation model including the scale and the weight accessibility also heavily determine the architectural failure modes introduced in the previous sections. Identical plan-execute-synthesize architectures exhibit qualitatively different failure patterns when instantiated using small open weight models vis-à-vis large proprietary systems [74, 76]. Existing research agents treat the underlying language model as a black box and assume that increased scale suffices for autonomous research [41]. However empirical evaluations show that model choice shifts failure location rather than reducing failure incidence. Smaller models fail during planning due to insufficient decomposition while larger models fail during synthesis through confident hallucination. While increased scale alters, it does not eliminate these architectural limitations [22, 76].

The parameter capacity paradox: Open-source LLMs in the 7B–13B range generally lag behind commercial models on agent-style tasks, with prior benchmarks reporting substantially higher hallucination on factual question answering and weaker reliability in multi-step settings [22, 74]. Prior work attributes much of this gap to capacity limits; smaller models have weaker reasoning and working memory, which makes it harder to sustain consistent beliefs across long agent dialogues and increases the chance of fabricated intermediate steps [74]. Model scaling reduces these errors, but size alone is not decisive. Across benchmarks, improvements within a model family from scaling can be large, yet cross-family comparisons suggest that architecture, data, and instruction tuning can matter as much as raw parameter count [22]. Moreover, end-to-end evaluations that require reproducing machine learning papers from scratch show that even strong proprietary models and research-agent systems frequently fail to deliver complete, correct results, with common breakdowns in experimental design, methodological rigor, and execution reliability [76]. Smaller models often fail in the planning module because they cannot keep many constraints in mind, so they break a complex goal into overly simple sub-tasks and miss key requirements [42, 74]. Larger models make fewer planning mistakes, but they can still produce a clean and confident plan that is wrong and looks correct at first glance [42]. In the execution module, research agents run for a long time (about 46,000 seconds per task vs. 250 seconds for reasoning-only agents), so early constraints get lost during coding and debugging [76]. This leads to "constraint drop" bugs, such as forgetting the intended cross-validation setup, and smaller models degrade more under long runs due to limited context [40, 74]. In the synthesis module, scale does not prevent hallucinations: reviewers can accept plausible but wrong abstracts, and even frontier models still hallucinate on technical QA, especially on multi-hop or conflicting evidence [22, 46, 73].

The open weight verification tradeoff: Closed-weight proprietary models (e.g., GPT-4, Claude 3.5 Sonnet) often outperform open-weight models but their closed design prevents inspection or modification of the reasoning process, which makes auditing, verification, and reproducibility difficult for autonomous research. In contrast, open-weight models can be fine-tuned to become competitive in low-data, low-resource, domain-specific settings, while remaining easier to adapt and control. They support stronger safety and governance hooks such as privacy-preserving training, mechanistic interpretability and better abstention behavior [71].

Module-specific failure propagation: Systematic evaluations of research agents report high failure rates across the full pipeline. In an analysis of 1,642 multi-agent traces, 44.2% of failures were specification errors introduced during planning, and execution failures ranged from 41% to 86.7% depending on the architecture and